

Internship Training Report

On

"Android App Development"

Submitted in partial fulfillment of the requirements for the award of Degree of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTER APPLICATIONS

SESSION — 2025-26

Submitted By:

Name : Gunjan Vishwakarma

Roll Number : 24CEBMC606

Semester : IV

Department : MCA

Submitted To:

Dr. Rakesh Poonia (HOD of MCA)

Engineering College Bikaner (Raj.) 334004

Engineering College Bikaner, Pugal Road
RIICO Karni Industrial Area, Bikaner, Rajasthan-334004
Telephone: +91-1512253404

Certificate



CDTS
Cloud & DevOps

Certificate of Completion

This is to certify that Mr. Gunjan Vishwakarma S/o Sh. Chander Shekhar Vishwakarma, student of MCA from Engineering College Bikaner, Bikaner has successfully completed an industrial training cum internship program on “**Android App Development**” and also worked on the same project. The duration of internship was 45 days from 19.01.2026 to 05.03.2026 During the internship session his behaviour was good.

NARENDRA SINGH BHUI
Academy Director

Certificate number: **CDTS/CERT/Gen-File/249**

Issued date: **05.03.2026**

URL: <https://www.cdtsbikaner.in> Contact: +91-7742163889

📍 Behind Rajasthan Patrika Press, Amar Singh Pura, Bikaner PIN 334001 (Rajasthan) INDIA

Society Reg. No. COOP/2020/BIKANER/200308

Acknowledgement

I would like to express my sincere gratitude to Centre for Cloud & DevOps Training Society (CDTS), Bikaner for providing me the opportunity to complete my internship training on Android App Development. The organization provided excellent practical exposure and an industry-level learning environment throughout the internship period.

I am thankful to the Director Mr. Narendra Singh Bhui, mentors, and trainers at CDTS Bikaner for their valuable guidance, support, and encouragement. Their guidance helped me understand Android development workflows, AR integration using Unity and ARCore, and mobile application deployment.

I would also like to express my sincere gratitude to Dr. Rakesh Poonia (HOD of MCA Department, Engineering College Bikaner) and Dr. Ankur Goswami (Faculty Guide, MCA Department) for their constant academic guidance and support throughout the training period.

I am grateful to Engineering College Bikaner and Bikaner Technical University for providing the infrastructure and academic environment necessary for completing this industrial project.

I would also like to thank my family and friends for their continuous motivation and support. The internship helped me improve my technical knowledge, problem-solving abilities, and confidence in mobile application development.

.....

(Signature)

Gunjan Vishwakarma

Roll No: 24CEBMC606 | 4th Semester

Master of Computer Applications

Engineering College Bikaner (Raj.) 334004

Candidate's Declaration

I, Gunjan Vishwakarma, hereby declare that the Internship Training Report entitled "**Android App Development**" submitted to the Department of Computer Application, Engineering College Bikaner, affiliated to Bikaner Technical University (BTU), Rajasthan, in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications, is a record of original work carried out by me under the guidance of Dr. Ankur Goswami, MCA Department, Engineering College Bikaner.

I further declare that this report has not been submitted to any other university or institution for the award of any degree or diploma.

.....

(Signature)

Gunjan Vishwakarma

Roll No: 24CEBMC606

Master of Computer Applications

Engineering College Bikaner (Raj.) 334004

Date: _____

Table of Contents

S. No.	Topic
1	Introduction
2	About the Organization
3	Objectives of Internship
4	Introduction to Android Development
5	Augmented Reality & Unity
6	AR Foundation & ARCore
7	C# Programming for Unity
8	Game Architecture & Design Patterns
9	Enemy Systems & Game AI
10	Physics & Collision Detection
11	User Interface Development
12	Audio System Implementation
13	Week Wise Internship Report
14	Project Work — Asteroid Impact AR
15	Challenges Faced
16	Skills Gained
17	Conclusion
18	References

1. Introduction

Industrial training is an essential component of technical education because it helps students gain practical exposure and understand real industry environments. Classroom learning provides theoretical knowledge, while industrial training bridges the gap between academic understanding and real-world implementation.

During this internship on Android App Development at Centre for Cloud & DevOps Training Society (CDTS), Bikaner, practical knowledge about Android application development, Unity game engine, Augmented Reality using ARCore, C# programming, and mobile deployment was gained. The internship provided hands-on exposure to industry-level tools and technologies used for mobile application development.

The primary project developed during this internship was **Asteroid Impact AR** — an Augmented Reality survival game for Android where players defend Earth from continuously spawning asteroids and UFO enemies within their real-world surroundings. The game was built using Unity 6 with AR Foundation and ARCore.

The internship also improved problem-solving abilities, project implementation skills, understanding of modular software architecture, and practical experience with AR-integrated mobile application development.

2. About the Organization

Centre for Cloud & DevOps Training Society (CDTS), Bikaner is a professional training organization that provides industrial training and certification programs in cloud computing, DevOps, Linux administration, Android development, and software engineering.

The organization focuses on practical learning and project-based implementation. Students are provided with hands-on sessions and real-time examples to improve technical understanding and industry readiness. CDTS Bikaner promotes open-source technologies and practical learning approaches that help students prepare for industrial careers.

Detail	Information
Organization	Centre for Cloud & DevOps Training Society (CDTS)
Location	Amar Singh Pura, Behind Rajasthan Patrika Press, Bikaner-334001
Director	Narendra Singh Bhui
Training Domain	Android App Development
Internship Duration	19 January 2026 to 05 March 2026 (45 Days)
Registration	Society Reg No: COOP/2020/BIKANER/200308

The technologies provided during training include Android development, Unity Engine, AR/VR development, cloud computing, DevOps tools, and software project implementation. The organization provides excellent infrastructure and mentorship for student development.

3. Objectives of Internship

The main objectives of this internship were:

- To understand Android application development workflow and tools
- To gain practical knowledge of Unity 6 game engine and C# programming
- To learn Augmented Reality concepts using AR Foundation and ARCore
- To understand mobile game architecture and modular component design
- To implement real-time gameplay systems including enemy spawning, collision detection, and health management
- To understand ScriptableObject-based state management in Unity
- To gain experience in Android APK building and device deployment
- To improve troubleshooting and problem-solving skills in a mobile development environment
- To develop a complete project from concept to deployment as a practical deliverable
- To improve communication, teamwork, and project management skills

4. Introduction to Android Development

Android is the world's most widely used mobile operating system, powering billions of devices globally. Android application development involves creating software applications for devices running the Android OS. Applications can be built using various frameworks and engines depending on the purpose of the application.

For game and interactive AR application development, Unity is the most widely adopted cross-platform engine. Unity provides a complete development environment including physics simulation, rendering pipelines, audio systems, animation, and AR integration — all exportable to Android as an APK.

Key Android Development Concepts

The following core concepts were studied during the internship:

- Android APK — the application package format used to distribute and install apps on Android devices
- Android SDK — Software Development Kit providing tools, APIs, and libraries for Android development
- Gradle Build System — used to compile and package Android applications
- ARCore — Google's augmented reality SDK for Android enabling motion tracking, environmental understanding, and light estimation
- Unity Android Build Settings — configuration for targeting Android devices including minimum API level, graphics API selection, and IL2CPP scripting backend

Advantages of Android Platform

- Largest global user base with broad device accessibility
- Open-source platform allowing deep customization
- Google Play Store for application distribution
- Strong AR support through ARCore
- Wide range of supported hardware including AR-capable devices

5. Augmented Reality & Unity

Augmented Reality (AR) is an interactive technology that superimposes computer-generated objects onto the real world in real time, as seen through a device camera. Unlike Virtual Reality which replaces the environment entirely, AR enhances the physical world with virtual elements.

Unity 6 is a real-time development platform widely used for game development, simulation, and AR/VR applications. Unity supports AR development through the AR Foundation framework, which provides a unified API layer that works across ARCore (Android) and ARKit (iOS).

Features of Unity for AR Development

- AR Foundation — cross-platform AR framework abstracting ARCore and ARKit
- AR Session — manages the AR lifecycle including initialization and tracking
- AR Session Origin — defines the coordinate space for AR content
- AR Camera — renders both the camera feed and virtual objects together
- Plane Detection — identifies horizontal and vertical surfaces in the environment
- Environmental Tracking — maintains spatial awareness during device movement

AR Gaming Applications

AR gaming transforms physical environments into interactive game worlds. Players interact with virtual characters and objects while remaining aware of their real surroundings. This creates a unique sense of immersion not possible with traditional screen-based games.

Popular examples include Pokémon GO, which pioneered mainstream AR gaming, and various spatial games built using ARCore that place game elements within real indoor and outdoor environments.

6. AR Foundation & ARCore

AR Foundation is Unity's official framework for building augmented reality applications. It provides a device-agnostic API that developers use to implement AR features, with platform-specific implementation handled by provider packages — ARCore for Android and ARKit for iOS.

ARCore Capabilities

Feature	Description
Motion Tracking	Tracks device position and orientation in 3D space using the camera and IMU sensors
Environmental Understanding	Detects flat surfaces such as floors and tables for AR content placement
Light Estimation	Estimates real-world lighting conditions to render virtual objects with realistic lighting
Depth API	Provides depth data for occlusion effects where real objects can appear in front of virtual ones
Augmented Faces	Enables face tracking for AR effects overlaid on the user's face

AR Foundation Components Used in Project

- AR Session — initialized on game scene load to begin AR tracking
- AR Session Origin — defined the world coordinate origin for all virtual objects
- AR Camera — provided camera feed and rendered virtual enemies within physical space
- ViewportPointToRay — used from screen center for accurate touch-based shooting alignment with the AR camera

7. C# Programming for Unity

C# (C-Sharp) is a modern, object-oriented programming language developed by Microsoft. It is the primary scripting language used in Unity for game logic, component behavior, and system implementation. C# provides strong typing, memory management through garbage collection, and excellent object-oriented capabilities.

Key C# Concepts Applied

Concept	Application in Project
MonoBehaviour	Base class for all Unity scripts; provides lifecycle methods such as Start(), Update(), and OnTriggerEnter()
Coroutines (IEnumerator)	Used for time-based logic such as enemy spawn intervals, shoot cooldowns, and UFO state transitions
UnityEvents	Decoupled event system used to trigger visual effects, audio, and score updates without direct references
ScriptableObject	GameState implemented as a ScriptableObject to act as a shared data store across all gameplay systems
Interfaces & Components	TryGetComponent() used for safe, type-checked component retrieval during collision handling
Conditional Compilation	#if UNITY_EDITOR directives used to support both Editor (mouse) and mobile (touch) input in the same script

Scripts Developed During Internship

- PlayerMouseController.cs — handles touch/mouse input and raycasting for shooting
- GameManager.cs — initializes session variables and manages Time.timeScale
- GameState.cs — ScriptableObject acting as centralized state and event bus
- UFO.cs — state machine managing Idle, Attacking, and Cooldown states
- UFOAttack.cs — coroutine-based periodic shooting while UFO is active
- SpawnColliderArea.cs — coroutine loop generating random-position enemy spawns
- Health.cs — manages health values for Player and Earth entities
- MakeDamage.cs — applies damage on trigger collision using tag comparison
- UIController.cs — subscribes to GameState events and updates HUD elements

- `MoveForward.cs` — applies frame-rate-independent forward movement to asteroids
- `Rotate.cs` — applies randomized continuous rotation to asteroid objects
- `FlashEffect.cs` — provides visual damage feedback through material color interpolation
- `AudioSfx.cs` — `ScriptableObject`-based audio system with random clip selection
- `DestroyAfterTime.cs`, `DestroyObject.cs`, `DestroyOnGameOver.cs` — object lifecycle management

8. Game Architecture & Design Patterns

Modern game development relies on well-structured software architecture to maintain modularity, testability, and scalability. During the internship, several important design patterns and architectural approaches were studied and applied.

Modular Component-Based Architecture

Unity follows a component-based architecture where game objects are composed of individual script components, each handling a single responsibility. This follows the Single Responsibility Principle, making each script independently testable and reusable across different objects.

ScriptableObject Data Architecture

The `GameState` `ScriptableObject` pattern was used to create a shared data container and event bus that all gameplay systems can access without direct script-to-script references. This significantly reduces coupling between systems and makes the architecture easier to extend.

Event-Driven Communication

`UnityEvents` were used throughout the project to enable decoupled communication. For example, when Earth's health reached zero, `GameState.OnGameOver` fired, which `UIController` received to display the Game Over screen — without the health system knowing about the UI system at all.

State Machine Pattern

The UFO enemy system was implemented using a state machine with `Idle`, `Attacking`, and `Cooldown` states. State transitions triggered `UnityEvents` that activated or deactivated components, enabling clean separation of behavior by state.

9. Enemy Systems & Game AI

Enemy system design is a core component of game development. During the internship, two types of enemy systems were implemented — a spawner-based asteroid system and a state-machine-driven UFO system.

Asteroid Spawning System

The `SpawnColliderArea.cs` script uses a coroutine loop to continuously generate asteroid enemies at random positions within a `Box Collider`'s bounds. Three separate spawner instances handle large asteroids (5-second interval), medium asteroids (10-second interval), and small asteroids (8-second interval). The spawning loop runs while `GameState.GameOver` is false.

Enemy Type	Spawn Interval	Movement	Threat Level
Large Asteroid	5 seconds	Forward — high speed	High damage
Medium Asteroid	10 seconds	Forward — medium speed	Medium damage
Small Asteroid	8 seconds	Forward — lower speed	Low damage
UFO	State-based	Trajectory-driven	Periodic attack bursts

UFO State Machine

The UFO system implements a three-state behavior machine managed by UFO.cs. In the Idle state, the UFO is positioned off-screen at a very large coordinate. After a random cooldown duration, it transitions to Attacking, spawning near the player and moving through calculated trajectory vectors. After completing its trajectory, it returns to Idle. UFOAttack.cs enables periodic firing while the UFO is in the Attacking state.

10. Physics & Collision Detection

Unity's physics engine handles collision detection, rigidbody simulation, and trigger events. During the internship, the following physics and collision concepts were applied:

Raycasting

Raycasting is the process of projecting an invisible ray from a point in a direction and checking what it hits. In Asteroid Impact AR, raycasting is used for the shooting system. ViewportPointToRay() from the AR Camera's center point creates a ray that travels into the scene. If it hits a collider, the bullet is aimed at that hit point; otherwise, the bullet travels 50 units along the ray direction.

Trigger Collisions

OnTriggerEnter(Collider other) is called when two trigger colliders overlap. MakeDamage.cs uses this event to detect when an asteroid or UFO projectile contacts the Player or Earth, then retrieves the Health component and applies damage. This approach avoids direct references between enemy and target scripts.

Damage Pipeline

Source	Target	Effect
Asteroid collision	Player	Player health reduced
Asteroid collision	Earth	Earth health reduced
UFO projectile	Player	Player health reduced
UFO projectile	Earth	Earth health reduced
Player projectile	Asteroid/UFO	Enemy health reduced → destroyed

11. User Interface Development

Unity's Canvas system provides a flexible UI framework for creating HUDs, menus, and interactive overlays. TextMeshPro (TMP) is used for high-quality text rendering. During the internship, both the gameplay HUD and the Game Over screen were implemented.

Gameplay HUD Elements

UI Element	Component	Purpose
Score Display	TMP_Text	Shows real-time score updated by GameState events
Player Health Bar	UI Slider	Shows player health value updated by Health.cs
Earth Health Bar	UI Slider	Shows Earth health value; reaching 0 triggers Game Over
Crosshair	RectTransform	Centered on mobile; follows mouse in Editor

Event-Driven UI Updates

UIController.cs subscribes to GameState events in OnEnable() and unsubscribes in OnDisable() to prevent memory leaks. When OnIncreaseScore fires, UpdateScoreUI() updates the score text. When OnGameOver fires, ShowGameOverScreen() activates the Game Over panel and displays the final score.

12. Audio System Implementation

Audio is an essential component of game experience. During the internship, a ScriptableObject-based audio system was implemented to provide reusable, configurable audio management across all game objects.

AudioSfx.cs is a ScriptableObject that stores an array of AudioClips along with parameters including volume, pitch, loop flag, start delay, and spatial blend. PlayAudio(GameObject parent) attaches an AudioSource to the specified parent object, selects a random clip from the array, applies all parameters, and plays the audio. This approach supports randomized sound selection, which prevents repetitive audio and improves immersion.

Audio Features Implemented

- Random clip selection from a pool — prevents repetitive identical sounds
- Configurable volume and pitch per sound asset
- Spatial blend support for 3D positional audio on UFO sounds
- Loop support for ambient and continuous sounds
- PlayAudioOnAwake.cs for automatic playback on object creation
- StopAudio() for clean audio shutdown on UFO state transitions

13. Week Wise Internship Report

Week 1 — Introduction to Android & Unity

During the first week, an introduction to Android application development and the Unity game engine was provided. Unity 6 installation, project structure, GameObject and Component concepts, scene management, and basic C# scripting were practiced. The Unity Editor interface, Inspector panel, Hierarchy, and Project windows were explored. Basic MonoBehaviour lifecycle methods (Start, Update, Awake) were studied through practical exercises.

Week 2 — AR Foundation & ARCore Integration

In the second week, Augmented Reality concepts were introduced. AR Foundation framework installation, AR Session setup, AR Session Origin configuration, and AR Camera integration were practiced. ARCore capabilities including motion tracking and environmental understanding were studied. Practical sessions included setting up a basic AR scene that displayed virtual objects within the real-world camera feed.

Week 3 — Gameplay Systems — Player & Shooting

The third week focused on implementing core gameplay mechanics. The player controller was developed with touch and mouse input support using conditional compilation. The raycasting-based shooting system was implemented using ViewportPointToRay for accurate mobile targeting. The projectile system including instantiation, forward movement via MoveForward.cs, and automatic destruction via DestroyAfterTime.cs was completed.

Week 4 — Enemy Systems & Game AI

The fourth week focused on enemy system implementation. The SpawnColliderArea.cs coroutine-based spawning system was developed for all three asteroid types. The UFO.cs state machine with Idle, Attacking, and Cooldown states was implemented. Trajectory-based UFO movement using Vector3.MoveTowards and the UFOAttack.cs periodic firing system were completed. Asteroid rotation via Rotate.cs was also implemented.

Week 5 — Health, Score & UI Systems

During the fifth week, the health management system was implemented for both Player and Earth entities using Health.cs and MakeDamage.cs. The GameState ScriptableObject was created as the centralized state manager and event bus. UIController.cs was developed to subscribe to GameState events and update health bars and score displays. The Game Over screen with final score display and navigation buttons was completed.

Week 6 — Audio, Polish & Final Deployment

The final week focused on audio system implementation, visual effects, object lifecycle management, and Android APK deployment. AudioSfx.cs ScriptableObject-based audio was integrated. FlashEffect.cs damage visual feedback was implemented. DestroyOnGameOver.cs cleanup was added. The complete project was built as an Android APK, deployed to a real ARCore-capable device, and all functionality was verified through manual testing.

14. Project Work — Asteroid Impact AR

The main project developed during the internship was **Asteroid Impact AR** — an Android Augmented Reality survival game where players defend Earth from continuously spawning asteroids and UFO attacks within their real-world physical environment.

Project Overview

Parameter	Details
Project Name	Asteroid Impact AR
Platform	Android (ARCore-capable devices)
Engine	Unity 6
Language	C#
AR Framework	AR Foundation / ARCore
Game Type	Endless Survival AR Shooter
Total Scripts	23 C# scripts

Project Architecture

The project follows a modular component-based architecture with a ScriptableObject-based GameState serving as the central communication hub. Six primary modules handle distinct responsibilities: AR Module, Enemy Spawning Module, UFO Attack Module, Health Management Module, Score Management Module, and User Interface Module.

Project Workflow

- Step 1: Unity project created with AR Foundation and ARCore packages installed
- Step 2: Menu Scene built with Play and Quit navigation using MenuButtons.cs and SceneLoader.cs
- Step 3: Game Scene hierarchy set up with AR Session Origin, AR Camera, Player, Planet, Spawners, UFO, HUD, and GameManager
- Step 4: GameState ScriptableObject created with Score, GameSpeed, and GameOver fields with OnIncreaseScore and OnGameOver events
- Step 5: PlayerMouseController.cs implemented with dual input support and raycasting-based shooting
- Step 6: SpawnColliderArea.cs configured for three asteroid types with independent spawn intervals
- Step 7: UFO.cs state machine and UFOAttack.cs periodic firing system implemented
- Step 8: Health.cs and MakeDamage.cs connected to Player and Earth objects with tag-based collision handling
- Step 9: UIController.cs developed to subscribe to GameState events and update HUD elements
- Step 10: AudioSfx.cs ScriptableObject audio system and FlashEffect.cs visual feedback integrated
- Step 11: Android APK built with ARCore support enabled, deployed to real device, and tested

Testing Results

Test Case	Input	Expected Result	Status
Game Start	Press Play	Load Game Scene	Pass
AR Initialization	Launch Game Scene	AR Session established	Pass
Enemy Spawning	Start gameplay	Continuous enemy generation	Pass
Shooting System	Touch screen	Bullet generated correctly	Pass
Damage System	Enemy collision	Health reduced	Pass
Health Bar Update	Receive damage	Health bar updates	Pass
Score Update	Enemy eliminated	Score increments	Pass
Game Over Condition	Earth health = 0	Game Over screen displayed	Pass
Scene Navigation	Press Home button	Menu scene loaded	Pass
UFO State Transition	UFO completes path	Returns to Idle state	Pass

15. Challenges Faced

During the internship, the following challenges were encountered and resolved:

AR Camera Alignment for Shooting:

Ensuring that raycasting for the shooting system was correctly aligned with the AR Camera's viewport required switching from ScreenPointToRay to ViewportPointToRay from screen center, ensuring mobile shooting accuracy regardless of the player's camera angle.

UFO Positioning in AR Space:

Calculating appropriate trajectory positions for the UFO relative to the player's forward direction in the AR coordinate system required careful use of player transform forward vectors with randomized offsets to create natural-feeling attack paths.

Enemy Collision Behavior:

Unexpected collisions between projectiles and spawned enemies at certain angles required adjustment of collider sizes and physics layer settings to ensure accurate hit registration.

Decoupled System Communication:

Connecting independent systems such as the health manager, UI controller, and spawner without creating tight coupling required learning the ScriptableObject event pattern and UnityEvent-based communication.

Android Build Configuration:

Configuring the Unity project for Android with ARCore support required setting the correct scripting backend (IL2CPP), minimum API level (Android 7.0), and enabling the ARCore package in XR Plugin Management.

16. Skills Gained

The internship helped in developing the following technical and professional skills:

Technical Skills

- Unity 6 game engine — scene management, component architecture, and prefab system
- C# programming — coroutines, UnityEvents, ScriptableObjects, and component patterns
- Augmented Reality development — AR Foundation, ARCore, AR Session, and spatial tracking
- Android APK building and deployment — Unity Android Build Settings and device testing
- Game systems design — health management, enemy spawning, collision detection, and state machines
- UI development — Unity Canvas, TextMeshPro, Slider components, and event-driven UI updates
- Audio system implementation — ScriptableObject-based audio with random clip selection
- Modular architecture — applying Single Responsibility Principle and decoupled communication patterns

Professional Skills

- Project planning and iterative development — breaking a complex project into manageable weekly milestones
- Problem-solving and debugging — diagnosing and resolving build errors, collision issues, and AR positioning problems
- Technical documentation — writing clear code comments and maintaining project documentation
- Time management — completing all assigned modules and the final project within the 45-day internship period
- Communication and teamwork — working with mentors and trainers to clarify requirements and receive feedback

17. Conclusion

The internship on Android App Development at Centre for Cloud & DevOps Training Society (CDTS), Bikaner provided valuable industrial exposure and practical understanding of mobile application development, Unity game engine, and Augmented Reality technologies. The training significantly improved technical skills, problem-solving abilities, and project implementation confidence.

The primary deliverable — Asteroid Impact AR — is a fully functional Android AR survival game that demonstrates the successful integration of Unity 6, AR Foundation, ARCore, and a comprehensive set of gameplay systems including dynamic enemy spawning, touch-based shooting mechanics, health management, score tracking, and real-time UI feedback. All ten functional test cases passed on a real Android device.

The ScriptableObject-based GameState architecture developed during this project demonstrated the value of decoupled, event-driven system design in game development, a pattern applicable across many categories of software engineering beyond games.

The experience and skills gained during this internship — in AR integration, modular game architecture, component-based design, Android deployment, and technical project documentation — will be directly valuable in future academic projects and professional software development opportunities.

18. References

1. Unity Technologies. Unity 6 Documentation. <https://docs.unity3d.com/6000.0/Documentation/Manual/>
2. Google Developers. ARCore Overview and Developer Guide. <https://developers.google.com/ar>
3. Unity Technologies. AR Foundation Package Documentation. <https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@latest/>
4. Microsoft Documentation. C# Programming Language Reference. <https://docs.microsoft.com/en-us/dotnet/csharp/>
5. Unity Technologies. TextMeshPro Documentation. <https://docs.unity3d.com/Packages/com.unity.textmeshpro@latest/>
6. Unity Technologies. Unity Learn — Game Development Tutorials. <https://learn.unity.com/>
7. Unity Technologies. ScriptableObjects in Unity Manual. <https://docs.unity3d.com/Manual/class-ScriptableObject.html>
8. Unity Technologies. Physics.Raycast API Reference. <https://docs.unity3d.com/ScriptReference/Physics.Raycast.html>
9. Unity Technologies. UnityEvent API Reference. <https://docs.unity3d.com/ScriptReference/Events.UnityEvent.html>
10. Bikaner Technical University. MCA Programme Curriculum Guidelines. Session 2024-26.
11. Centre for Cloud & DevOps Training Society (CDTS), Bikaner. Industrial Training Completion Certificate. March 2026.
12. Engineering College Bikaner. MCA Department Project Guidelines. Session 2025-26.